

Respuesta Prueba Técnica — Seguros Bolívar

Documento original: `RESPUESTA_PRUEBA_TECNICA.md` · Anexo visual generado con Archify

Prueba Técnica – Desarrollador TI Seguros Bolívar

- **Autor:** Diego Alejandro Sepulveda Huetio
 - **Tecnología utilizada:** Desarrollo asistido con **Gemini 3.8**
 - **Stack del proyecto:** React 19 + Vite + TypeScript (frontend), Spring Boot 3 + Java 17 (backend real), Express/tsx (API en memoria que replica el contrato), H2 + JPA.
-

1. Cómo funciona el proyecto

Es una **plataforma de gestión de pólizas de arrendamiento** (individuales y colectivas) que responde a los 4 módulos de la prueba.

- **Módulo 1 – System Design:** Pestaña **System Design** (y README del backend con la arquitectura). Incluye diagrama de componentes (API Gateway, Servicio de Pólizas, Servicio de Riesgos, Bus de eventos, Adapter CORE WebLogic, Notificaciones, BD), 3 patrones justificados (Event-Driven, Hexagonal, CQRS + API Gateway), modelo de datos y NFRs (escalabilidad, logs/observabilidad, tolerancia a fallos y versionado de APIs).
- **Módulo 2 – API de Gestión de Pólizas (Spring Boot):** El backend `spring-boot-backend/` expone los 6 endpoints obligatorios con capas `controller` → `service` → `repository`, entidades `Poliza` y `Riesgo`, reglas de negocio y security filter `x-api-key`.
- **Módulo 3 – BBDD:** Pestaña **BBDD & Git**: DDL con constraints, consultas de negocio e índices; README backend con las 3 estrategias de optimización (índices cubrientes, particionamiento/vistas materializadas, CTE con filtrado temprano).
- **Módulo 4 – Git:** README backend y pestaña **BBDD & Git**: estrategia `git cherry-pick` justificada.
- **Frontend (React 19 + TypeScript):**
 - **Barra Superior (Header Operativo Exclusivo):**
 -  **Tablero Mensual:* KPIs de cartera, cánones activos, metas de renovación porcentual e historial mes a mes.
 -  **Pólizas & Riesgos:* Emisión, cancelación en cascada, administración de riesgos amparados y renovación con ajuste por IPC.
 -  **Recordatorios:* Alertas de renovación y pagos por canales Email, SMS y Push.
 -  **Consola WebLogic CORE:* Bitácora de eventos para auditoría de integración con la capa media.
 -  **Acceso directo a Instrucciones:* Botón ancla con scroll suave hacia el final del proyecto.
 - **Parte Inferior del Proyecto (Asistente Interactivo de Soporte & Evaluación Técnica):**

- 💡 ***Diseño Interactivo y No Invasivo:*** Para no saturar ni confundir al usuario durante los procesos operativos diarios, la sección inicia en un estado de consulta amigable que pregunta si necesita ayuda o desea ver la documentación técnica.
- 📌 **Sección Exclusiva: Consumo de Endpoints:** Guía técnica interactiva para aprender a consumir la REST API, especificación de contratos, autenticación `x-api-key: 123456`, ejemplos funcionales en **cURL**, **JavaScript/React (Fetch)**, **Spring Boot (WebClient)** y **Python (requests)**, junto con una **Consola Sandbox** de pruebas en vivo.
- 🏠 ***Módulo 1 – System Design:*** Diagrama de arquitectura, justificación de patrones y NFRs.
- ⚡ ***Módulos 3 & 4 – BBDD & Git:*** DDL con constraints, optimizaciones SQL y flujo Git.
- 🌐 ***Código Spring Boot:*** Explorador interactivo del código fuente Java.
- ✕ ***Botón de Cierre:*** Permite ocultar las instrucciones en cualquier momento con un clic para regresar al modo estrictamente operativo.

2. Verificación frente a los requisitos del PDF

Módulo 1 – Diseño de Sistema ✅

Requisito	Estado	Evidencia
Arquitectura de alto nivel	✅	<code>spring-boot-backend/README.md</code> (§ Módulo 1) y <code>src/components/SystemDesignView.tsx</code>
3 patrones de arquitectura justificados	✅	Event-Driven, Hexagonal (Ports & Adapters), CQRS + API Gateway
Modelo de datos principal	✅	Poliza, Riesgo, Auditoría Core Log, Recordatorio
Escalabilidad / Logs / Tolerancia a fallos / Versionado	✅	Sección NFR del System Design View
Diagrama de componentes	✅	Servicio Pólizas, Riesgos, Notificaciones, Adapter CORE, BD, API Gateway

Módulo 2 – Prueba Técnica Práctica ✅

Requerimiento	Estado	Dónde
GET <code>/polizas</code> (filtro por <code>tipo</code> y <code>estado</code>)	✅	<code>PolizaController</code> + <code>PolizaRepository.findByTipoAndEstado</code>
GET <code>/polizas/{id}/riesgos</code>	✅	<code>PolizaController</code> → <code>RiesgoService.obtenerRiesgosPorPoliza</code>
POST <code>/polizas/{id}/renovar</code> (+IPC, estado → RENOVADA)	✅	<code>PolizaServiceImpl.renovarPoliza</code> (canon/prima × (1+IPC/100))
POST <code>/polizas/{id}/cancelar</code>	✅	<code>PolizaServiceImpl.cancelarPoliza</code>
POST <code>/polizas/{id}/riesgos</code> (solo Colectiva)	✅	<code>RiesgoServiceImpl.agregarRiesgoAPoliza</code> con validación de tipo

Requerimiento	Estado	Dónde
POST /riesgos/{id}/cancelar	✓	RiesgoServiceImpl.cancelarRiesgo
Regla: individual = 1 riesgo	✓	Validación countByPolizaIdAndEstado >= 1 → 422
Regla: no renovar póliza cancelada	✓	BusinessException → 422
Regla: cancelar póliza cancela sus riesgos	✓	Cascada sobre riesgoRepository
Regla: agregar riesgo valida tipo de póliza	✓	TipoPoliza.INDIVIDUAL rechazado con más de 1 riesgo
Mock externo POST /core-mock/evento	✓	CoreMockController (logs reales SLF4J) + server.ts /api/core-mock/evento
Seguridad x-api-key: 123456 (401 si falta)	✓	ApiKeyAuthFilter (Spring) y middleware Express
Estructura Spring Boot por capas	✓	controller / service / impl / repository / model / dto / security / exception
Entidades básicas (Poliza, Riesgo)	✓	model/entity/Poliza.java, Riesgo.java
Código funcional de endpoints	✓	Compila con Java 17 + Maven (mvn spring-boot:run, puerto 8080)
Validaciones de negocio	✓	GlobalExceptionHandler (422 negocio, 404, 400 validación, 500)
README con instrucciones	✓	spring-boot-backend/README.md

Módulo 3 – BBDD ✓

Al menos 3 estrategias documentadas (consulta de orders 10M / customers 500K con filtro México):

1. Índices compuestos y cubrientes (Covering / Index-Only Scan).
2. Particionamiento de tablas + vista materializada con refresco concurrente.
3. Reestructuración de consulta con CTE/filtrado temprano + EXPLAIN ANALYZE.

Módulo 4 – Git ✓

git cherry-pick <hash> para traer el hotfix de seguridad de main a feature/new-login sin arrastrar el resto; justificado frente a merge y rebase.

3. Cómo instalar y ejecutar

A) Frontend + API en memoria (la app completa, puerto 3000)

```
# 1. Requisitos: Node.js 18+
npm install

# 2. (Opcional) .env.local con tu GEMINI_API_KEY
cp .env.example .env.local

# 3. Ejecutar (índice + API en memoria)
npm run dev
```

Abrir `http://localhost:3000`. Toda llamada a la API exige el header `x-api-key: 123456` (editable desde el botón gris del top bar para probar el 401).

La app funciona 100% con la API en memoria (`server.ts`). Nota: la clave de ambiente `GEMINI_API_KEY` solo se usa si agregas funciones del modelo; para la prueba técnica no es necesaria.

B) Backend Spring Boot real (puerto 8080)

```
cd spring-boot-backend

# Requisitos: Java 17+ y Maven 3.8+
mvn clean package
mvn spring-boot:run
```

- API en `http://localhost:8080` (ej.: `GET http://localhost:8080/polizas?tipo=COLECTIVA&estado=ACTIVA`).
- Consola H2: `http://localhost:8080/h2-console` (JDBC `jdbc:h2:mem:polizasdb`, user `sa`, sin password).
- El costado "WebLogic CORE" se simula con `POST /core-mock/evento`, que registra en logs cada intento de sincronización.

Verificación rápida de los endpoints

```
curl -X GET "http://localhost:8080/polizas?tipo=COLECTIVA" -H "x-api-key: 123456"
curl -X POST "http://localhost:8080/polizas/1/renovar" -H "x-api-key: 123456" -H "Content-Type: application/json" -d '{"porcentajeIpc": 9.28}'
curl -X POST "http://localhost:8080/core-mock/evento" -H "x-api-key: 123456" -H "Content-Type: application/json" -d '{"evento":"ACTUALIZACION","polizaId":555}'
```

4. Observaciones / detalles

- El `README.md` raíz es el plantilla genérica de AI Studio; la documentación real de la solución está en `spring-boot-backend/README.md` y en este documento.
- El backend Spring Boot inicia con la BD H2 vacía (sin datos semilla); la demo con datos la provee la API en memoria del frontend, que replica el mismo contrato y reglas.
- Comandos de lint/typecheck disponibles: `npm run lint (tsc --noEmit)` y `npm run build`.

Anexo — Arquitectura del proyecto (capturas de Archify)

A continuación, capturas generadas con **Archify** desde el documento `docs/seguros-polizas-architecture.html`. Cada vista resalta los componentes involucrados y explica cómo funciona esa parte del sistema.

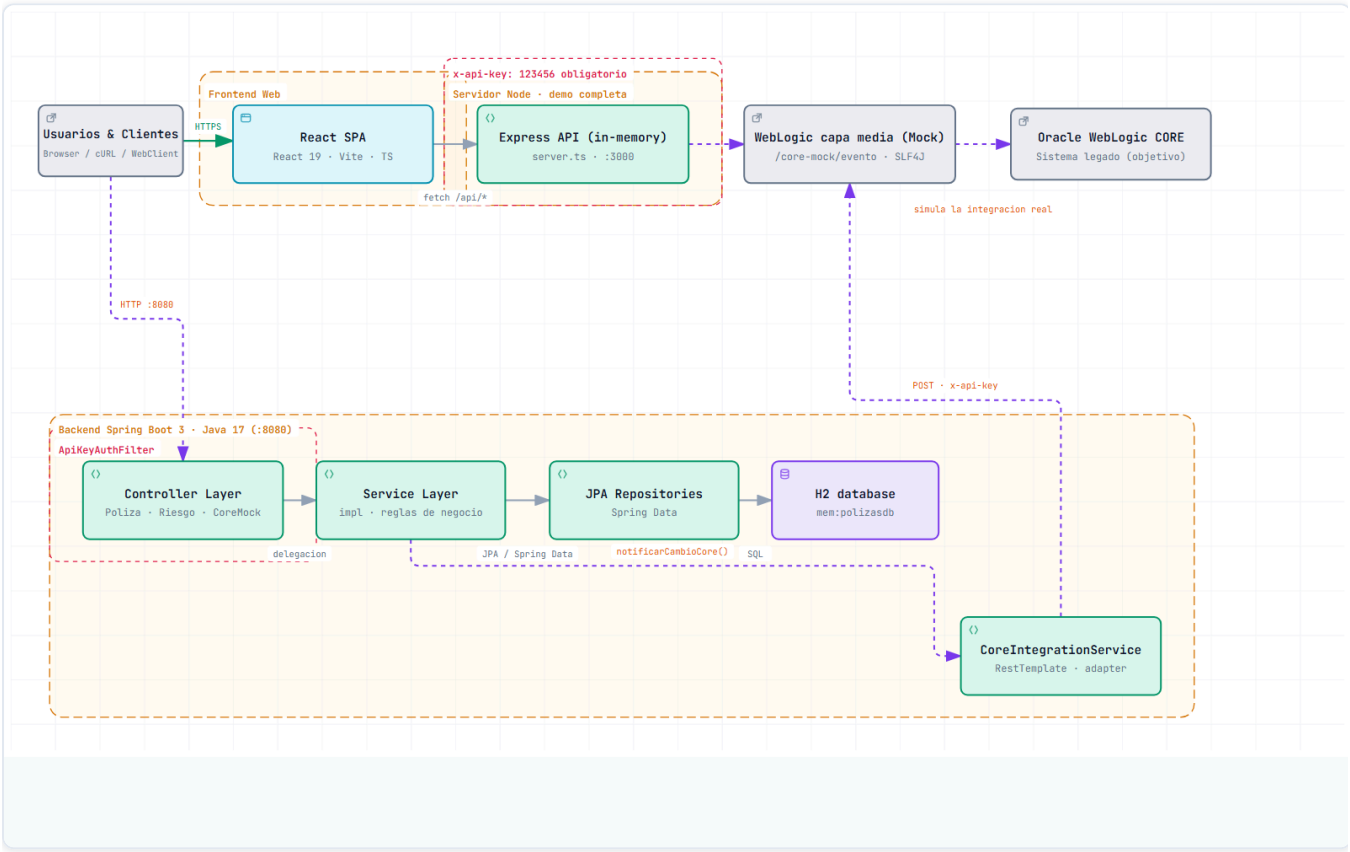


Figura 1. Vista general de la arquitectura SegurosPolizas (Archify). 10 componentes y 10 conexiones: frontend React 19, API demo Express en memoria, backend Spring Boot 3 (Controller → Service → Repository → H2) y la integración con la capa media WebLogic CORE.

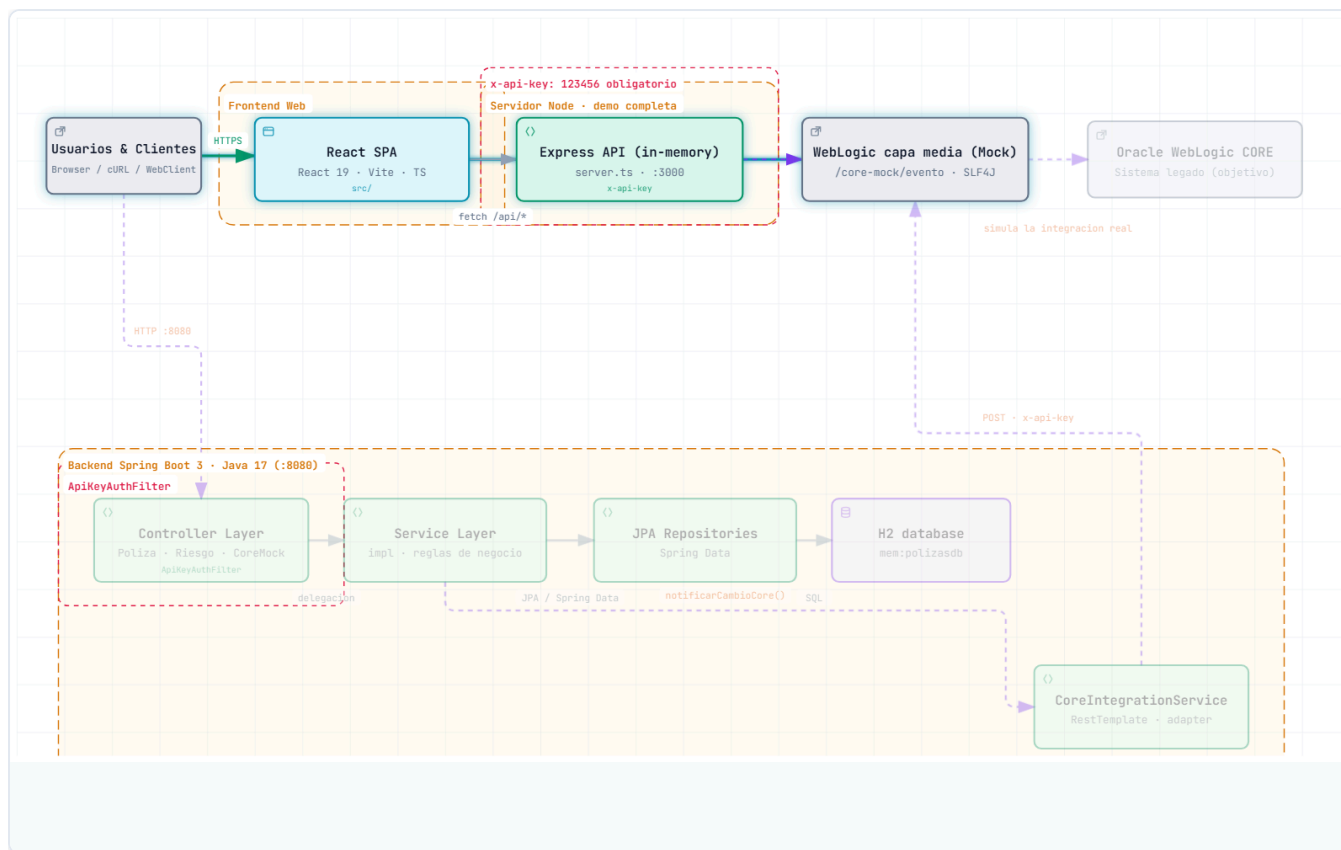


Figura 2. Vista "Ruta principal (demo)" — el camino operativo completo: Usuarios & Clientes → React SPA → Express API (:3000, API en memoria que replica el contrato Spring) → WebLogic capa media (mock). Cada mutación queda registrada como evento.

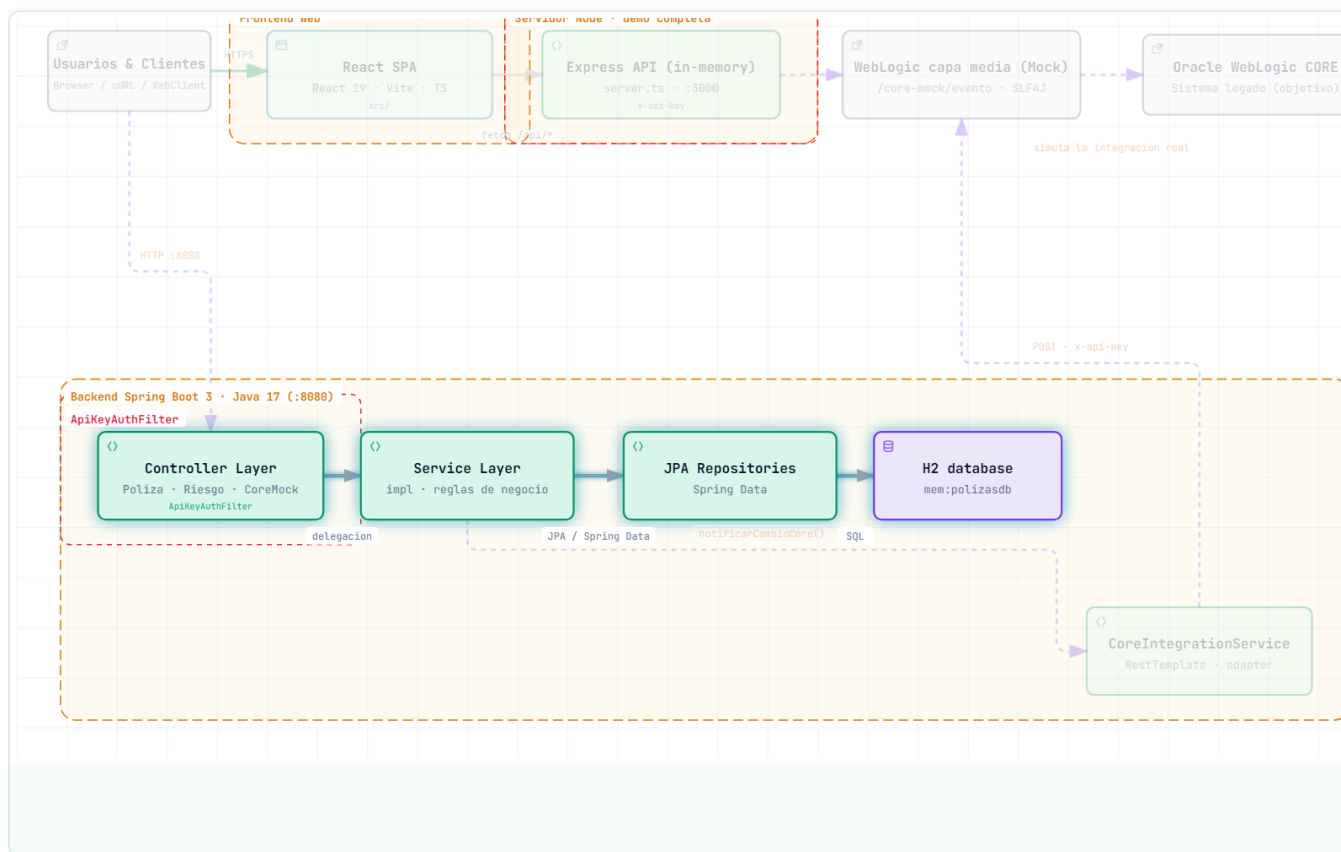


Figura 3. Vista "Backend real Spring Boot" — implementación Java 17 (:8080): Controller Layer (Poliza · Riesgo · CoreMock, filtro ApiKeyAuthFilter) → Service Layer (reglas de negocio) → JPA Repositories → H2 (mem:polizasdb).

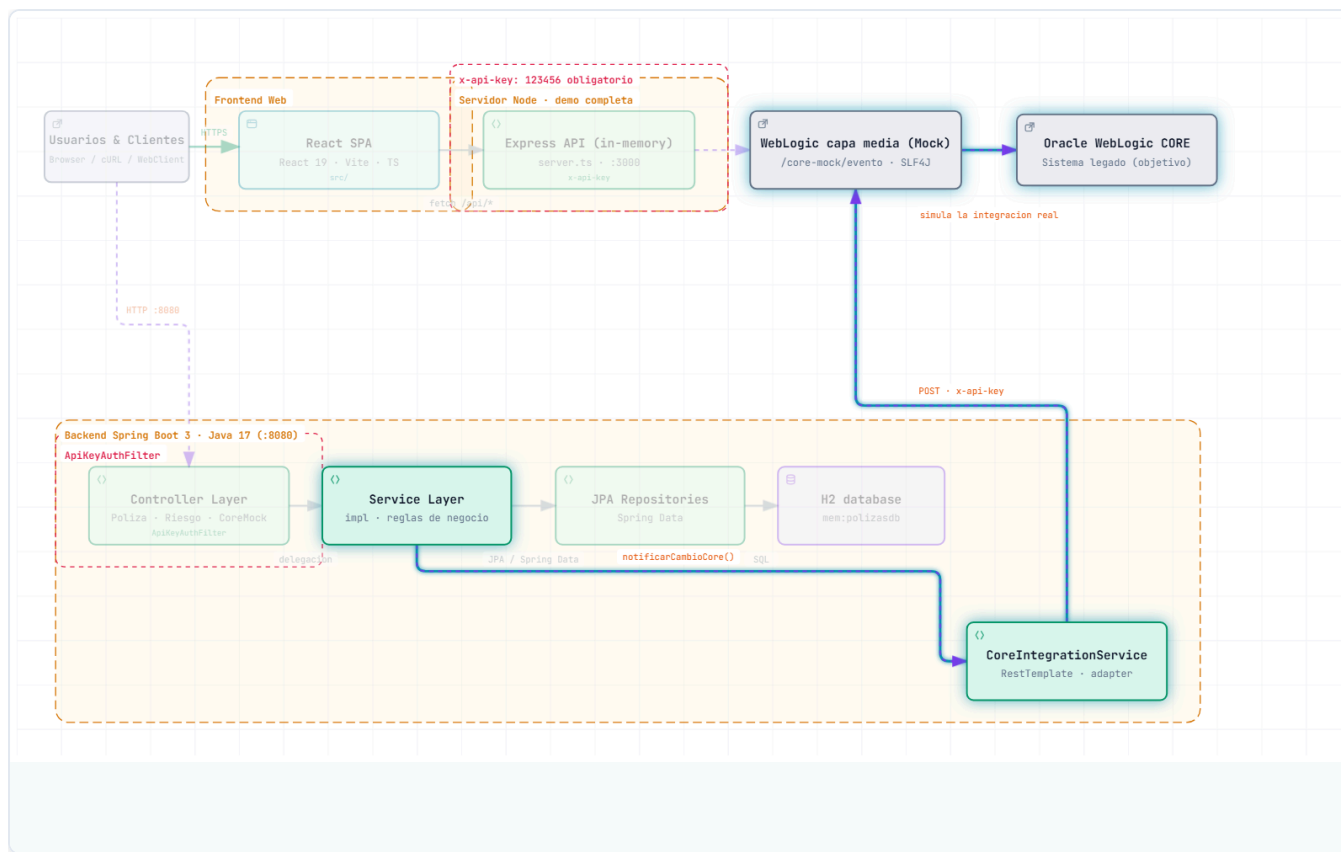


Figura 4. Vista "WebLogic CORE (Módulo 1)" — cómo funciona la integración: Service llama a CoreIntegrationService (adapter RestTemplate, patrón Hexagonal/Ports & Adapters + Outbox), que notifica a la capa media (mock /core-mock/evento con logs SLF4J) con destino al Oracle WebLogic CORE.

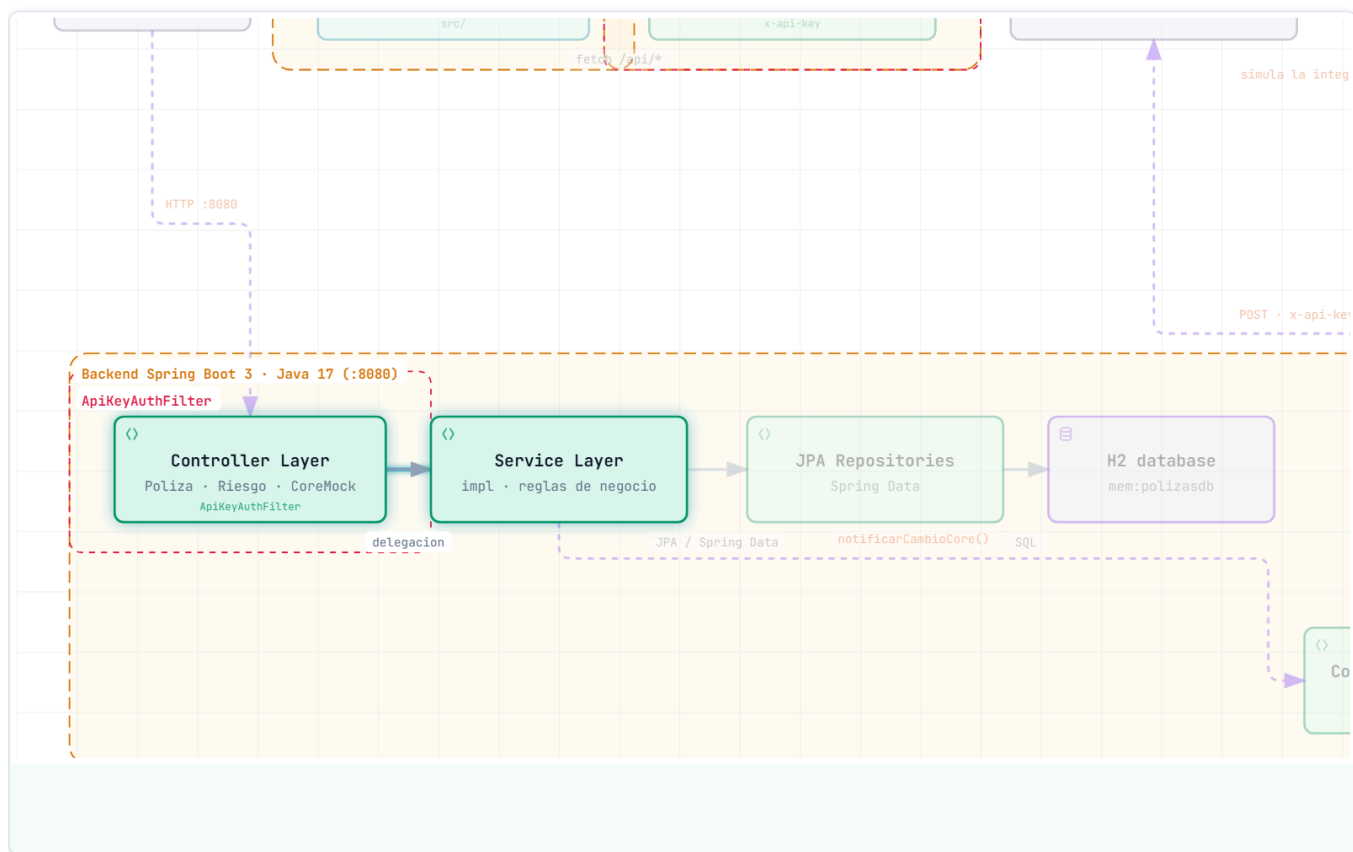


Figura 5. Vista "Reglas y endpoints (Módulo 2)" — autenticación x-api-key (401 si falta), los 6 endpoints obligatorios y las reglas de negocio con 422: individual = 1 riesgo, no renovar póliza CANCELADA, cancelación en cascada de riesgos.

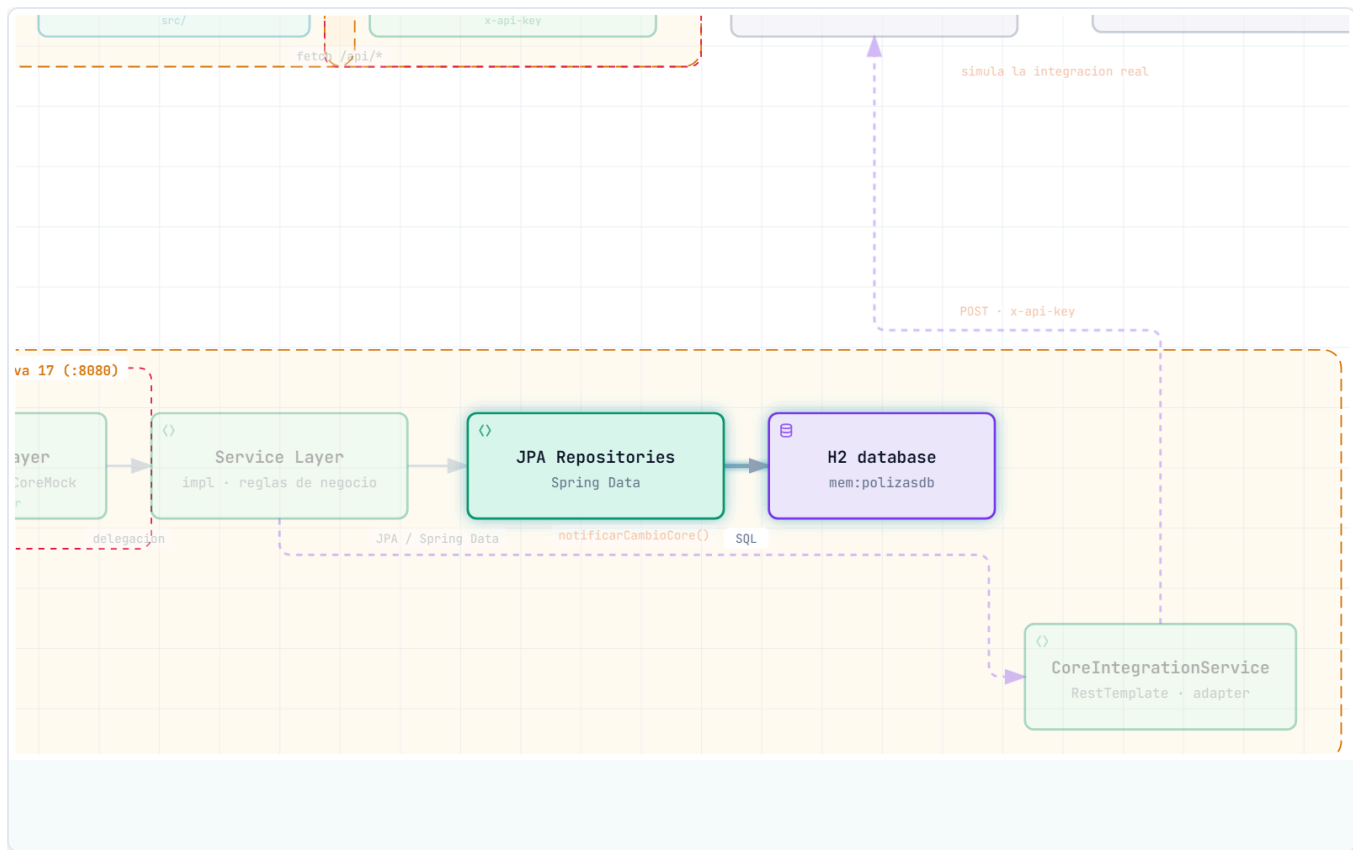


Figura 6. Vista "BBDD y Git (Módulos 3-4)" — estrategias SQL (índices cubrientes, particionado + vista materializada con refresco concurrente, CTE con filtrado temprano + EXPLAIN ANALYZE) y git cherry-pick del hotfix de main a feature/new-login.